



Pandas

O que é?

- ▶ Ferramenta de **manipulação e análise de dados**;
- ▶ Construída sobre a linguagem de programação **Python**;
- ▶ Open source.



Que tipo de dados pandas gerencia?

Importar o pandas

- ▶ Para começar a trabalhar com o pandas, basta importar o pacote;
- ▶ O alias padrão para o pandas é pd.

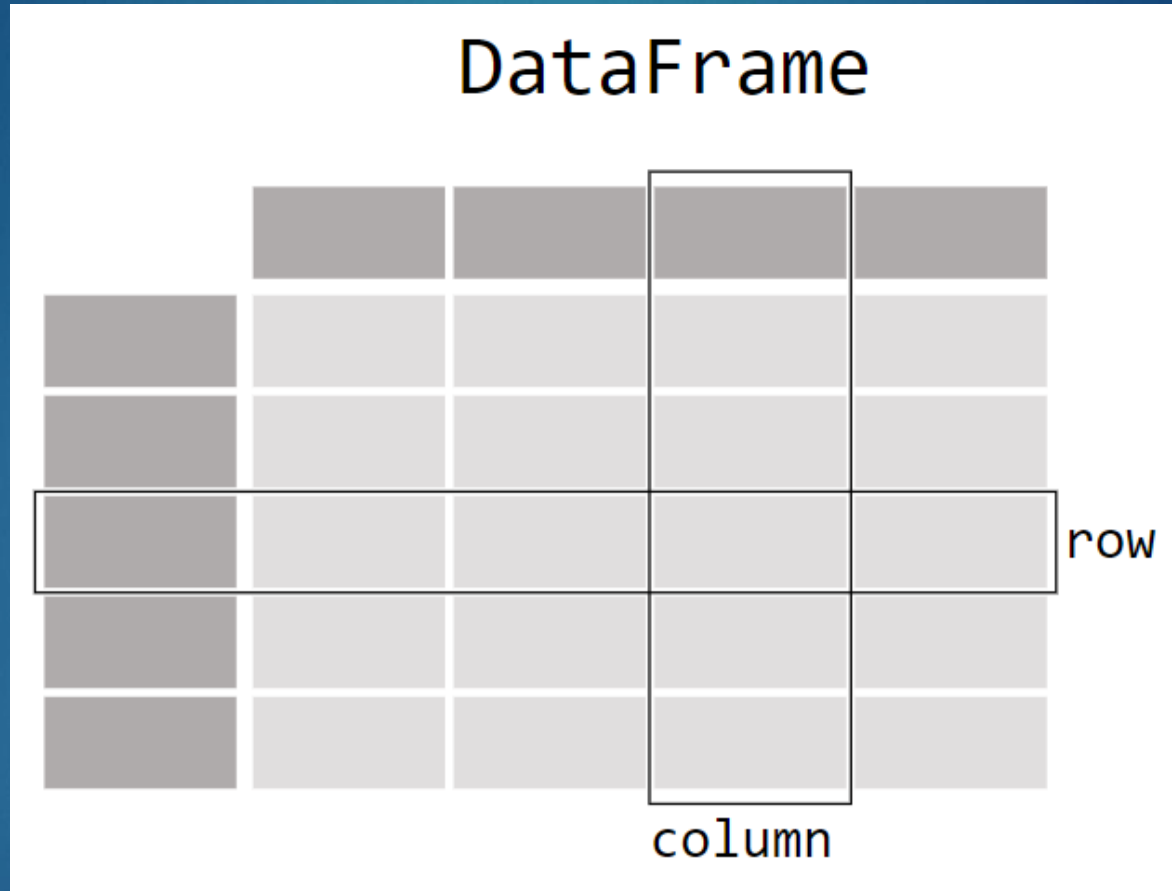
```
In [1]: import pandas as pd
```

Todas as imagens foram retiradas do site: <https://pandas.pydata.org/>

DataFrame

- ▶ **Estrutura bidimensional que armazena dados** de diferentes tipos;
- ▶ Os dados são organizados em linhas e colunas;
- ▶ As colunas possuem rótulos que indicam qual dado será armazenado (ex.: nome, idade, sexo);
- ▶ As linhas trazem os dados referentes a um registro.

100



DataFrame

```
In [2]: df = pd.DataFrame(  
    ...:     {  
    ...:         "Name": [  
    ...:             "Braund, Mr. Owen Harris",  
    ...:             "Allen, Mr. William Henry",  
    ...:             "Bonnell, Miss. Elizabeth",  
    ...:         ],  
    ...:         "Age": [22, 35, 58],  
    ...:         "Sex": ["male", "male", "female"],  
    ...:     }  
    ...: )  
    ...:
```

```
In [3]: df
```

```
Out[3]:
```

	Name	Age	Sex
0	Braund, Mr. Owen Harris	22	male
1	Allen, Mr. William Henry	35	male
2	Bonnell, Miss. Elizabeth	58	female

Séries (Series)

- ▶ **Cada coluna em um DataFrame é uma Série;**
- ▶ Utilizado quando se quer trabalhar com os dados de apenas uma coluna (ex.: idade);
- ▶ Para selecionar uma coluna, coloque o nome da coluna entre colchetes.

```
In [4]: df["Age"]  
Out[4]:  
0      22  
1      35  
2      58  
Name: Age, dtype: int64
```


Séries (Series)

- ▶ Você pode criar Séries do zero:

```
In [5]: ages = pd.Series([22, 35, 58], name="Age")
```

```
In [6]: ages
```

```
Out[6]:
```

```
0    22
```

```
1    35
```

```
2    58
```

```
Name: Age, dtype: int64
```

Uso de métodos

- ▶ Métodos podem ser utilizados em DataFrames e Séries para diversos fins;
- ▶ Para saber a idade máxima armazenada basta:

```
In [7]: df["Age"].max()  
Out[7]: 58
```

```
In [8]: ages.max()  
Out[8]: 58
```

describe()

- Método que provê mais informações sobre dados numéricos em DataFrame.

```
In [9]: df.describe()
```

```
Out[9]:
```

	Age
count	3.000000
mean	38.333333
std	18.230012
min	22.000000
25%	28.500000
50%	35.000000
75%	46.500000
max	58.000000

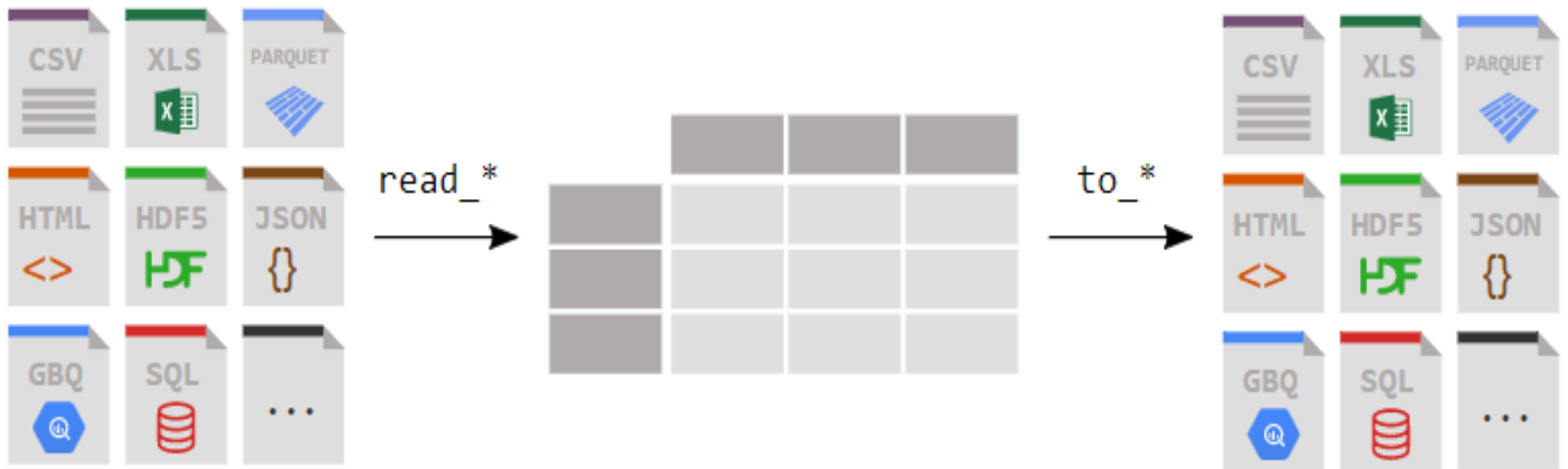
Resumo

- ▶ Importar o pacote: **import pandas as pd**;
- ▶ Uma tabela de dados é armazenada como **DataFrame**;
- ▶ Cada coluna no DataFrame é uma série (**Series**);



Como ler e escrever dados tabulares?

Ler e escrever dados tabulares



Ler dados de arquivos CSV

- ▶ A função **read_csv()** lê dados de arquivos CSV e os armazena em um DataFrame;

```
In [2]: titanic = pd.read_csv("data/titanic.csv")
```

Ler dados

- ▶ Ao ler um DataFrame, as primeiras e as últimas 5 linhas são exibidas por padrão:

```
In [3]: titanic
```

```
Out[3]:
```

	<i>PassengerId</i>	<i>Survived</i>	<i>Pclass</i>	<i>...</i>	<i>Fare</i>	<i>Cabin</i>	<i>Embarked</i>
0	1	0	3	...	7.2500	NaN	S
1	2	1	1	...	71.2833	C85	C
2	3	1	3	...	7.9250	NaN	S
3	4	1	1	...	53.1000	C123	S
4	5	0	3	...	8.0500	NaN	S
..	...	...	...	...	...	...	...
886	887	0	2	...	13.0000	NaN	S
887	888	1	1	...	30.0000	B42	S
888	889	0	3	...	23.4500	NaN	S
889	890	1	1	...	30.0000	C148	C
890	891	0	3	...	7.7500	NaN	Q

```
[891 rows x 12 columns]
```


head()

- Utilizado para exibir as **primeiras N linhas**:

```
In [4]: titanic.head(8)
```

```
Out[4]:
```

	<i>PassengerId</i>	<i>Survived</i>	<i>Pclass</i>	<i>...</i>	<i>Fare</i>	<i>Cabin</i>	<i>Embarked</i>
0	1	0	3	...	7.2500	NaN	S
1	2	1	1	...	71.2833	C85	C
2	3	1	3	...	7.9250	NaN	S
3	4	1	1	...	53.1000	C123	S
4	5	0	3	...	8.0500	NaN	S
5	6	0	3	...	8.4583	NaN	Q
6	7	0	1	...	51.8625	E46	S
7	8	0	3	...	21.0750	NaN	S

```
[8 rows x 12 columns]
```

tail()

- ▶ Método utilizado para exibir as **últimas N linhas** de um DataFrame.
- ▶ Ex.: `dataframe.tail(10)`

dtypes

- ▶ Mostra como o pandas interpreta cada tipo de dado de coluna;
- ▶ Tipos de dados no pandas:
 - ❑ inteiros (**int64**)
 - ❑ floats (**float64**)
 - ❑ strings (**object**)

```
In [5]: titanic.dtypes
Out[5]:
PassengerId      int64
Survived          int64
Pclass           int64
Name             object
Sex              object
Age             float64
SibSp            int64
Parch            int64
Ticket           object
Fare             float64
Cabin            object
Embarked         object
dtype: object
```

Gravar dados

- ▶ Para gravar dados, utiliza-se métodos com o padrão **to_*** (* = tipo do arquivo);
- ▶ **to_excel** - gravar dados em um arquivo excel:

```
In [6]: titanic.to_excel("titanic.xlsx", sheet_name="passengers", index=False)
```

info()

- Método que fornece mais informações acerca do DataFrame:

```
In [9]: titanic.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   PassengerId     891 non-null   int64
 1   Survived        891 non-null   int64
 2   Pclass         891 non-null   int64
 3   Name           891 non-null   object
 4   Sex            891 non-null   object
 5   Age           714 non-null   float64
 6   SibSp          891 non-null   int64
 7   Parch          891 non-null   int64
 8   Ticket         891 non-null   object
 9   Fare           891 non-null   float64
10  Cabin          204 non-null   object
11  Embarked       889 non-null   object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Resumo

- ▶ Ler dados: **read_***
- ▶ Exportar dados: **to_***
- ▶ Métodos **head/tail/info** e atributos **dtypes** são convenientes para uma primeira verificação.



Selecionar um subconjunto de um DataFrame

Selecionar colunas de um DataFrame



Selecionar colunas de um DataFrame

- ▶ Cada coluna de um DataFrame é uma Series;
- ▶ O objeto retornado de uma única coluna selecionada é uma Series;

```
In [6]: type(titanic["Age"])  
Out[6]: pandas.core.series.Series
```

shape

- **Atributo** que retorna o **nº de linhas e colunas** de uma Series ou DataFrame.

```
In [7]: titanic["Age"].shape  
Out[7]: (891,)
```

Selecionar várias colunas

- ▶ Basta utilizar colchetes dentro colchetes.

```
In [8]: age_sex = titanic[["Age", "Sex"]]
```

```
In [9]: age_sex.head()
```

```
Out[9]:
```

	Age	Sex
0	22.0	male
1	38.0	female
2	26.0	female
3	35.0	female
4	35.0	male

Filtrar linhas específicas

- Para selecionar apenas passageiros com mais de 35 anos:

```
In [12]: above_35 = titanic[titanic["Age"] > 35]
```

```
In [13]: above_35.head()
```

```
Out[13]:
```

	<i>PassengerId</i>	<i>Survived</i>	<i>Pclass</i>	<i>...</i>	<i>Fare</i>	<i>Cabin</i>	<i>Embarked</i>
1	2	1	1	...	71.2833	C85	C
6	7	0	1	...	51.8625	E46	S
11	12	1	1	...	26.5500	C103	S
13	14	0	3	...	31.2750	NaN	S
15	16	1	2	...	16.0000	NaN	S

```
[5 rows x 12 columns]
```

Filtrar linhas específicas

- Selecionar apenas passageiros em cabines de 1ª e 2ª classes:

```
In [16]: class_23 = titanic[titanic["Pclass"].isin([2, 3])]
```

```
In [17]: class_23.head()
```

```
Out[17]:
```

	<i>PassengerId</i>	<i>Survived</i>	<i>Pclass</i>	<i>...</i>	<i>Fare</i>	<i>Cabin</i>	<i>Embarked</i>
0	1	0	3	...	7.2500	NaN	S
2	3	1	3	...	7.9250	NaN	S
4	5	0	3	...	8.0500	NaN	S
5	6	0	3	...	8.4583	NaN	Q
7	8	0	3	...	21.0750	NaN	S

```
[5 rows x 12 columns]
```

notna()

- Função condicional que retorna true apenas quando o valor da linha não é nulo.

```
In [20]: age_no_na = titanic[titanic["Age"].notna()]
```

```
In [21]: age_no_na.head()
```

```
Out[21]:
```

	<i>PassengerId</i>	<i>Survived</i>	<i>Pclass</i>	<i>...</i>	<i>Fare</i>	<i>Cabin</i>	<i>Embarked</i>
0	1	0	3	...	7.2500	NaN	S
1	2	1	1	...	71.2833	C85	C
2	3	1	3	...	7.9250	NaN	S
3	4	1	1	...	53.1000	C123	S
4	5	0	3	...	8.0500	NaN	S

```
[5 rows x 12 columns]
```

Selecionar linhas e colunas específicas

- ▶ **loc** – selecionar linhas e colunas específicas.
- ▶ Selecionar apenas os **nomes** de passageiros com **mais de 35 anos**.

```
In [23]: adult_names = titanic.loc[titanic["Age"] > 35, "Name"]
```

```
In [24]: adult_names.head()
```

```
Out[24]:
```

```
1      Cumings, Mrs. John Bradley (Florence Briggs Th...
6                McCarthy, Mr. Timothy J
11             Bonnell, Miss. Elizabeth
13             Andersson, Mr. Anders Johan
15             Hewlett, Mrs. (Mary D Kingcome)
Name: Name, dtype: object
```


Selecionar linhas e colunas específicas

- **iloc** – selecionar linhas e colunas específicas indicando as posições na tabela.

```
In [25]: titanic.iloc[9:25, 2:5]
```

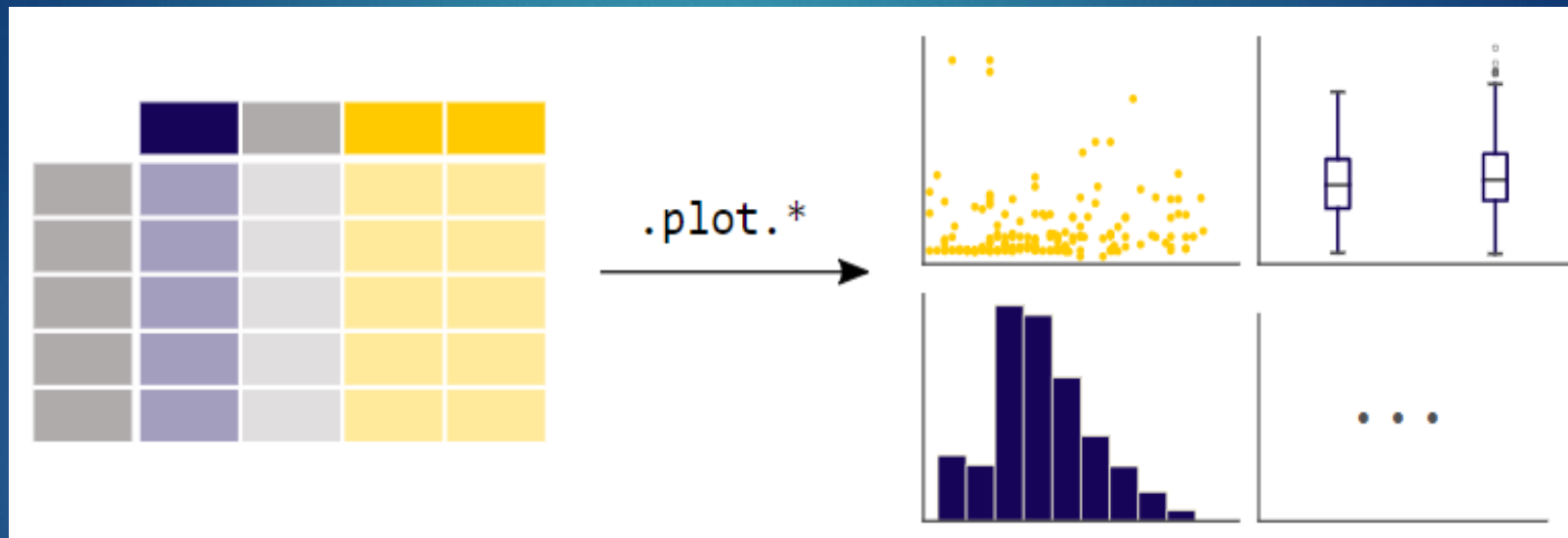
```
Out[25]:
```

	<i>Pclass</i>	<i>Name</i>	<i>Sex</i>
9	2	Nasser, Mrs. Nicholas (Adele Achem)	female
10	3	Sandstrom, Miss. Marguerite Rut	female
11	1	Bonnell, Miss. Elizabeth	female
12	3	Saunderscock, Mr. William Henry	male
13	3	Andersson, Mr. Anders Johan	male
..	...	...	...
20	2	Fynney, Mr. Joseph J	male
21	2	Beesley, Mr. Lawrence	male
22	3	McGowan, Miss. Anna "Annie"	female
23	1	Sloper, Mr. William Thompson	male
24	3	Palsson, Miss. Torborg Danira	female



Plotar com o pandas

Plotar com pandas



Plotar com pandas

- ▶ É preciso importar a biblioteca matplotlib.

```
In [1]: import pandas as pd
```

```
In [2]: import matplotlib.pyplot as plt
```

```
In [3]: air_quality = pd.read_csv("data/air_quality_no2.csv", index_col=0, parse_dates=True)
```

```
In [4]: air_quality.head()
```

```
Out[4]:
```

	station_antwerp	station_paris	station_london
datetime			
2019-05-07 02:00:00	NaN	NaN	23.0
2019-05-07 03:00:00	50.5	25.0	19.0
2019-05-07 04:00:00	45.0	27.7	19.0
2019-05-07 05:00:00	NaN	50.4	16.0
2019-05-07 06:00:00	NaN	61.9	NaN

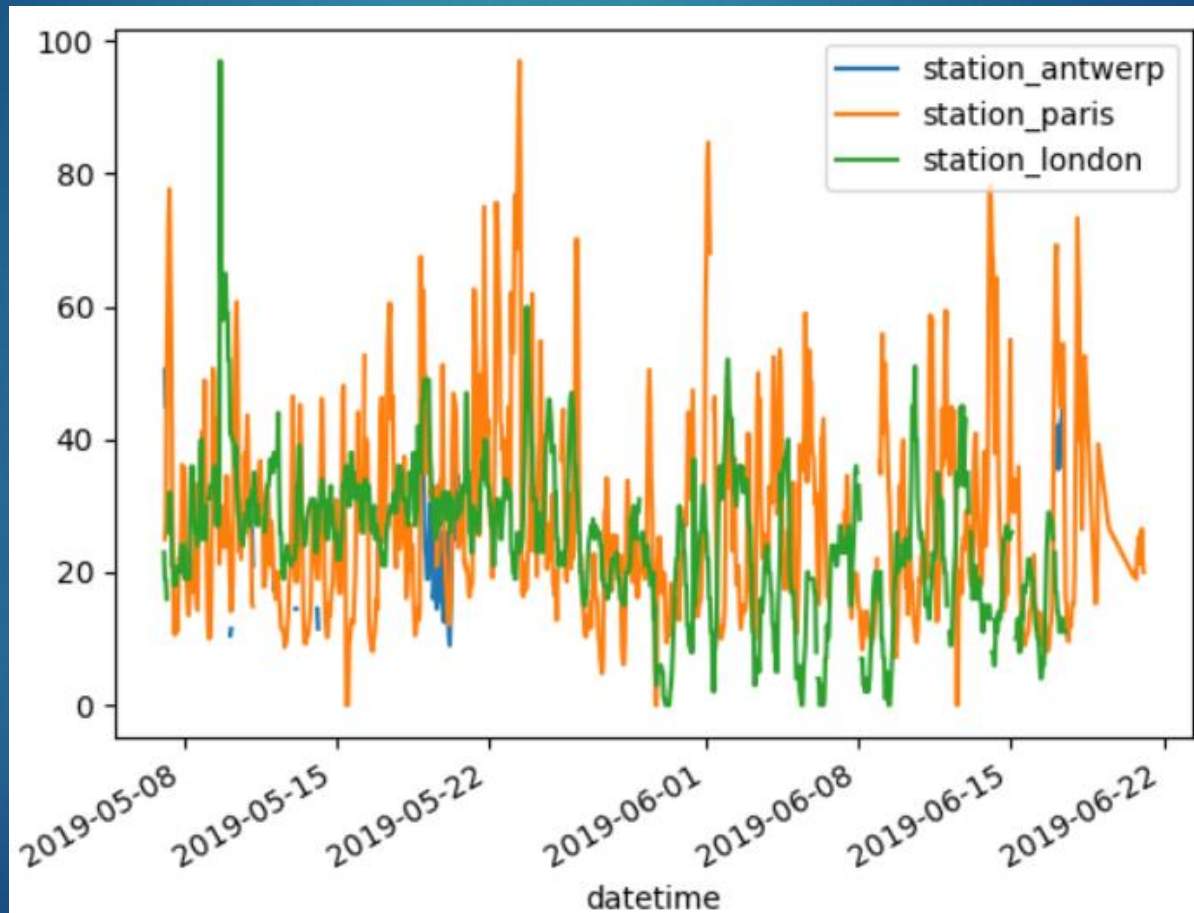
plot()

- ▶ Método que realiza a plotagem;
- ▶ Funciona com Series e DataFrame.

```
In [5]: air_quality.plot()  
Out[5]: <AxesSubplot: xlabel='datetime'>
```

plt.show()

- Exibe o resultado da plotagem.



Referências

- ▶ pandas.pydata.org. Getting started tutorials.
Disponível em:
https://pandas.pydata.org/docs/getting_started/intro_tutorials/index.html